

编译技术

实验设计实现报告

林子淇

目录

一、 参考编译器分析	1
1.1 核心设计理念	1
1.2 对本项目的启示	1
二、 编译器总体设计	1
2.1 管道架构	2
2.2 阶段接口设计	2
2.3 工程组织与数据流	2
三、 词法分析	2
3.1 正则引擎驱动的设计	3
3.2 贪婪匹配与优先级控制	3
3.3 扫描与错误处理	3
四、 语法分析	4
4.1 无类型 CST 设计	4
4.2 Parser Combinator 实现	4
4.3 错误恢复与回溯机制	4
五、 语义分析	5
5.1 AST 建模与视图转换	5
5.2 符号表管理	5
5.3 类型检查与错误诊断	5
六、 代码生成	6
6.1 SSA IR 构建	6
6.2 图着色寄存器分配	6
6.3 栈帧布局与函数调用	7
七、 代码优化	7
7.1 优化体系架构	7
7.2 SSA 构造与 Mem2Reg	7
7.3 数据流与控制流优化	8

一、参考编译器分析

本项目参考了 Typst 编译器的前端架构，引入“无损语法树”与“双视图”设计，以增强编译器的容错能力并支持潜在的工具链扩展。

1.1 核心理念

1.1.1 无损语法树

Typst 编译器采用 **无损语法树 (Lossless Syntax Tree)** 作为核心中间表示，保留源码中的空白字符与注释。其节点设计为无类型结构，允许任意节点嵌套。这种设计允许解析器在源码存在语法错误时，仍能生成包含“错误节点”的完整树结构，而非直接中断编译。同时，保留的格式信息使得语法高亮、自动格式化等 IDE 功能可以直接基于语法树实现，无需额外的解析 pass。

1.1.2 双视图架构

为了兼顾灵活性与类型安全，Typst 引入了“双视图”架构。**语法层 (Syntax Layer)** 基于无类型树，服务于 IDE 工具与增量解析，侧重于“源码的物理结构”。**计算层 (Compute Layer)** 则在无类型树之上构建强类型视图 (Typed View)，负责语义求值与代码生成。该层通过包装底层节点提供类型安全的访问接口，若底层结构不符合预期，该层会拦截错误，从而将语法校验延迟到语义分析阶段。

1.1.3 增量解析与节点追踪

为了支持高效的增量编译，Typst 摒弃了基于字节偏移量的定位方式，转而采用 Span Number 机制。每个语法节点被分配一个在多次编译间保持稳定的唯一标识符。这种设计不仅解决了文本编辑导致的偏移量失效问题，还极大地优化了缓存局部性，使得编译器能够快速定位并重用未变更的语法结构。

1.2 对本项目的启示

受此启发，本项目在前端设计中构建 CST (Concrete Syntax Tree) 作为无损的语法层，随后通过 Builder 模式将其转换为强类型的 AST 进行语义分析。这种解耦设计降低了 Parser 的复杂度，同时为后续可能引入的错误恢复机制预留空间。

二、编译器总体设计

本编译器采用多阶段 (Multi-stage) 管道架构，将编译流程解耦为若干独立且可测试的单元。该设计遵循关注点分离 (Separation of Concerns) 原则，为后续引入优化 Pass 和多后端支持提供扩展接口。

2.1 管道架构

编译器的核心驱动逻辑封装于 `Pipeline` 类中, 维护有序的阶段映射表以协调数据流动。这种链式组织方式允许根据配置动态调整执行路径。例如, 当 `CompilerConfig.OPTIMIZE` 开启时, 优化阶段会被插入到 IR 生成与代码生成之间。

```
public class Pipeline {
    public Pipeline() {
        // 注册各阶段, 支持按需配置优化 Pass
        stages.put("lexer", new LexerStage());
        stages.put("parser", new ParserStage());
        stages.put("semantic", new SemanticStage());
        stages.put("llvm", new IrGenStage());
        if (CompilerConfig.OPTIMIZE) {
            stages.put("optimize", new OptimizeStage());
        }
        stages.put("mips", new MipsGenStage());
    }
}
```

代码 1 编译器管道配置

2.2 阶段接口设计

为了规范各阶段的输入输出契约, 定义了泛型接口 `CompilerStage<I, A>`。该接口通过泛型约束确保阶段间数据流动的类型一致性。`ErrorReporter` 作为独立参数传递, 允许各阶段将错误诊断信息统一收集。`StageResult` 记录类型同时封装了结构化产物和文本报告, 满足不同场景的需求。

```
@FunctionalInterface
public interface CompilerStage<I, A> {
    // 统一的阶段处理接口, 分离产物与错误报告
    StageResult<A> process(I input, ErrorReporter errorReporter);
    record StageResult<A>(A artefact, String report) {}
}
```

代码 2 编译器阶段接口定义

2.3 工程组织与数据流

基于上述架构, 编译器被划分为三个逻辑层次。**前端 (Frontend)** 负责源码的结构化与语义校验, 包含 `Lexer`, `Parser`, `Semantic` 阶段。**中端 (Middle-end)** 负责中间代码生成与机器无关优化, 包含 `IrGen` 和 `Optimize` 阶段。**后端 (Backend)** 负责目标代码生成, 即 `MipsGen` 阶段。这种分层逻辑界定了各模块的职责边界, 使得中端和后端可以独立进行算法迭代。

三、词法分析

词法分析器 (`Lexer`) 负责将原始的字符流转换为具有语法意义的词法单元 (`Token`) 序列。

3.1 正则引擎驱动的设计

为了简化 DFA 的构造过程，本项目利用 Java 内置的 `java.util.regex` 引擎实现词法扫描。核心思想是将所有词法规则编译为一个巨大的正则表达式，利用“命名捕获组” (Named Capturing Group) 功能直接判定 Token 类型。

```
static {  
    // 利用 LinkedHashMap 保持插入顺序，确保最长匹配优先  
    TOKEN_PATTERNS.put("COMMENT", "//[^\n]*|/\\*.?*\\*/");  
    TOKEN_PATTERNS.put("LEQ", "≤"); // 优先于 "<"  
    TOKEN_PATTERNS.put("LSS", "<");  
    // ...  
    // 编译为命名捕获组正则: (?<COMMENT>...)|(?<LEQ>...)|...  
    TOKEN_PATTERN = Pattern.compile(combinedPattern.toString(), Pattern.DOTALL);  
}
```

代码 3 正则模式合并策略

3.2 贪婪匹配与优先级控制

在构建组合正则时，模式的顺序至关重要。由于正则引擎通常采用“最左最长”或“顺序优先”的匹配策略，必须显式处理前缀冲突。例如，对于输入 `≤`，如果 `<` 的规则定义在 `≤` 之前，引擎可能会优先匹配 `<`。因此，在 `LinkedHashMap` 中严格控制插入顺序，确保长词法单元优先于短词法单元被匹配。

3.3 扫描与错误处理

`scan()` 方法通过 `Matcher.find()` 循环遍历源码。每次匹配成功后，通过 `findMatchedGroup` 方法反查触发匹配的组名，从而确定 Token 类型。此外，词法分析器还承担了行号追踪的任务，通过统计换行符为每个 Token 标记准确的行号。对于非法字符，Lexer 会记录错误并尝试进行简单的错误恢复。

```
public TokenStream scan() {  
    while (matcher.find()) {  
        String groupName = findMatchedGroup(matcher);  
        switch (groupName) {  
            // 统计行号，用于错误报告  
            case "WHITESPACE", "COMMENT" -> line += countNewLines(lexeme);  
            // 区分标识符与关键字  
            case "IDENFR" -> addToken(KEYWORDS.getDefault(lexeme,  
TokenTypes.IDENFR), lexeme);  
            default -> addToken(TokenType.valueOf(groupName), lexeme);  
        }  
    }  
    return new TokenStream(tokens);  
}
```

代码 4 词法扫描主循环

四、语法分析

语法分析器负责将线性的 Token 流转换为具有层次结构的具体语法树 (CST)。本项目采用了基于组合子 (Combinator) 的递归下降分析法。

4.1 无类型 CST 设计

为了最大化前端的容错能力，设计了无类型的 CST 节点结构。所有语法结构均统一继承自 `CstNode` 抽象类。这种设计允许在解析阶段构建出包含“错误节点”或“不完整结构”的语法树，具体的节点类型仅作为数据的容器，其语义解释推迟到后续的 AST 构建阶段进行。

```
public abstract class CstNode {  
    // 统一的 Visitor 接口, 支持后续的 AST 构建与格式化  
    public abstract <T> T accept(CstVisitor<T> visitor);  
}
```

代码 5 CST 节点基类

4.2 Parser Combinator 实现

为了降低文法到代码的映射复杂度，引入了函数式编程中的组合子模式。通过定义 `seq`, `or`, `opt`, `many` 等高阶函数，将 EBNF 文法直接翻译为 Java 方法调用链。这种声明式的编写方式提高了代码的可读性，使得文法的修改变得简单。

```
private void initializeRules() {  
    // CompUnit → { Decl | FuncDef } MainFuncDef  
    define(CompUnit, seq(many(or(rule(FuncDef), rule(Decl))), rule(MainFuncDef)));  
  
    // ConstDecl → 'const' BType ConstDef { ',' ConstDef } ';' ;  
    define(ConstDecl, seq(  
        term(CONSTTK),  
        rule(BType),  
        rule(ConstDef),  
        many(seq(term(COMMA), rule(ConstDef))),  
        term(SEMICN, "i") // 错误码 "i" 用于缺失分号时的报错  
    ));  
}
```

代码 6 语法规则定义示例

4.3 错误恢复与回溯机制

为了处理语法错误并支持尝试性解析，引入了 `BacktrackMgr`。当 Parser 遇到不匹配的情况时，利用回溯机制恢复到之前的状态。同时，针对常见的语法错误，在 `term` 组合子中集成了 Panic Mode 恢复策略。当预期的终结符未找到时，记录相应的错误码并假定该符号存在，从而继续解析后续内容。

五、语义分析

语义分析阶段负责将松散的 CST 转换为语义严谨的抽象语法树，并在此过程中执行类型检查与符号解析。

5.1 AST 建模与视图转换

为了支持复杂的语义操作，设计了一套强类型的 AST 节点层次结构。`AstBuilder` 实现了 `CstVisitor` 接口，通过遍历 CST 节点将其重组为 AST。例如，在处理 `BinaryExpr` 时，Builder 会自动根据运算符优先级调整树结构，将扁平的 CST 序列转换为嵌套的二叉树形式。

```
public class AstBuilder implements CstVisitor<Object> {
    @Override
    public Object visit(NonTerm node) {
        // 根据 CST 节点类型分发构建逻辑
        return switch (node.type()) {
            case CompUnit → buildCompUnit(node);
            case FuncDef → buildFuncDef(node);
            // ...
            default -> null;
        };
    }
}
```

代码 7 CST 到 AST 的转换

5.2 符号表管理

为了正确处理 SysY 语言的块级作用域规则，设计了树状结构的符号表。每个 `Scope` 对象维护一个符号映射表，并持有指向父级作用域的引用。这种设计天然支持了变量遮蔽 (Shadowing) 机制：查找操作总是从当前作用域开始，逐级向上回溯，从而保证了内层定义的优先级高于外层。

```
public class SymbolTable {
    public void enterScope() {
        // 创建新作用域，指向当前作用域作为父级
        currentScope = new Scope(currentScope, ++scopeCounter);
    }

    public Symbol lookup(String name) {
        return currentScope.lookup(name); // 递归向上查找
    }
}
```

代码 8 符号表的作用域链管理

5.3 类型检查与错误诊断

`TypeChecker` 是语义分析的核心组件，它通过访问者模式遍历 AST，执行关键检查。这包括检测变量重定义和未定义使用，校验赋值语句左右两侧类型一致性，以及确保非 `void` 函数的

所有路径都有返回值等控制流检查。在遍历过程中，`TypeChecker` 还会将解析出的符号信息回填到 AST 节点中，为后续的中间代码生成提供必要的上下文信息。

六、代码生成

代码生成阶段分为两个子阶段：首先将 AST 转换为与机器无关的 LLVM IR，然后将 IR 映射为目标机器的 MIPS 汇编代码。

6.1 SSA IR 构建

我们设计了基于静态单赋值 (SSA) 形式的线性 IR。为了简化 SSA 的构造过程，`IrBuilder` 采用了 `Alloca` 指令为局部变量分配栈空间，从而暂时规避了 ϕ 节点的直接插入。这种策略生成的 IR 虽然包含冗余的 `Load / Store` 指令，但降低了前端实现的复杂度。后续的 `Mem2Reg` 优化 Pass 将负责把这些栈变量提升为虚拟寄存器。

```
public Value visit(VarDecl node) {
    // 为局部变量分配栈空间，后续由 Mem2Reg 提升为寄存器
    AllocaInst alloca = new AllocaInst(irType, node.name, null);
    // ...
}
```

代码 9 基于 `Alloca` 的变量声明翻译

6.2 图着色寄存器分配

为了最大化 MIPS 物理寄存器的利用率，实现了基于 Chaitin-Briggs 算法的图着色寄存器分配器 `GraphColoringRegAlloc`。该算法首先计算每个虚拟寄存器的活跃范围并构建冲突图。随后，移除度数小于 K 的节点并压栈。若无法简化，则根据启发式规则选择溢出代价最小的节点进行溢出。最后，按出栈顺序为节点分配物理寄存器。此外，引入了寄存器合并 (Coalescing) 优化，尝试消除 `Move` 指令连接的两个节点间的冲突，从而减少拷贝指令的数量。

```
public void allocate() {
    build(); // 构建冲突图
    makeWorklist(); // 初始化工作表

    // 迭代执行简化、合并、冻结与溢出选择
    while (!simplifyWorklist.isEmpty() || !worklistMoves.isEmpty() ||
           !freezeWorklist.isEmpty() || !spillWorklist.isEmpty()) {
        if (!simplifyWorklist.isEmpty()) simplify();
        else if (!worklistMoves.isEmpty()) coalesce(); // 寄存器合并
        else if (!freezeWorklist.isEmpty()) freeze();
        else selectSpill();
    }
    assignColors();
}
```

代码 10 图着色主循环

6.3 栈帧布局与函数调用

`MipsBuilder` 负责管理函数调用的栈帧布局。为了兼容 MIPS 调用规约, 栈帧被划分为参数区 (存放溢出的参数)、保存区 (保存被调用者保存的寄存器) 和局部变量区 (存放溢出的虚拟寄存器和数组)。在生成函数序言和尾声时, `Builder` 会自动计算栈帧大小, 并生成相应的指令来管理栈指针和恢复现场。

七、代码优化

为了提升生成代码的执行效率, 设计了基于 `PassManager` 的中端优化框架。该框架支持“分析”与“变换”的解耦, 并采用迭代收敛的策略调度各类优化 `Pass`。

7.1 优化体系架构

`PassManager` 维护了一个有序的 `Pass` 列表, 涵盖了从预处理、SSA 构造到标量优化、循环优化的完整流程。为了应对 `Pass` 之间的相互依赖, 管理器采用 `while(changed)` 循环机制: 只要任意 `Pass` 对 IR 进行了修改, 就会触发下一轮迭代, 直至 IR 状态收敛或达到最大迭代次数。

Pass 流水线拓扑图

图 1 Pass 流水线拓扑图

7.2 SSA 构造与 Mem2Reg

`Mem2Reg` 是中端优化的基石, 其核心任务是将栈上的局部变量 (`Alloca` 指令) 提升为虚拟寄存器, 从而消除冗余的内存访问。该算法首先识别仅被 `Load / Store` 指令使用的 `Alloca` 变量。接着, 基于支配边界 (Dominance Frontier) 理论, 在变量定义的汇合点插入 ϕ 节点。最后, 对支配树进行先序遍历, 维护每个变量的定义栈, 将 `Load` 替换为栈顶的最新定义。

```
private boolean runOnFunction(Function function) {
    // 1. 计算支配树与支配边界
    DominatorAnalysis.DominatorInfo domInfo =
    DominatorAnalysis.computeDominators(function);
    // 2. 在支配边界插入 Phi 节点
    insertPhiNodes(candidates, promotable, domInfo.dominanceFrontier());
    // 3. 遍历支配树进行重命名
    rename(entry, stacks, ...);
    return true;
}
```

代码 11 Mem2Reg 核心流程

7.3 数据流与控制流优化

在 SSA 形式的基础上, 实现了一系列经典的数据流优化。全局值编号 (GVN) 通过哈希表记录表达式的计算结果, 消除重复计算。稀疏条件常量传播 (SCCP) 结合控制流信息, 同时传播常量值并标记不可达代码。死代码消除 (DCE) 基于活跃性分析, 递归删除无副作用且未被使用的指令。此外, `SimplifyCfgPass` 负责清理控制流图中的冗余结构, 如合并单一后继的基本块、消除空的跳转块等, 从而减少分支指令的开销。